



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У НОВОМ
САДУ



Исидора Мићковић

PRACTICAL BYZANTINE FAULT TOLERANCE АЛГОРИТАМ

Семинарски рад из предмета

Примењени алгоритми у управљачким системима

Нови Сад, 2026.

Садржај

PBFT АЛГОРИТАМ	1
1. Увод и мотивација проблема консензуса	3
2. Опис PBFT алгоритма	3
2.1 Фазе алгоритма	4
3. Архитектура имплементираног симулатора	5
4. Верификована безбедносна својства	6
5. Позната ограничења и свесна поједностављења	7
6. Закључак	8
7. Литература	9

1. Увод и мотивација проблема консензуса

Дистрибуирани системи се све више ослањају на репликацију података и сервиса преко више независних чворова ради постизања високе доступности. Основни проблем који се притом јавља је консензус – како обезбедити да сви исправни чворови у систему донесу исту одлуку (нпр. изврше исте операције истим редоследом), чак и у присуству грешака.

Класичне технике репликације (Paxos, Viewstamped Replication) претпостављају бенигне грешке – чвор који откаже једноставно престаје да ради или пропушта кораке. Ова претпоставка није реална у системима изложеним софтверским грешкама, грешкама оператера или злонамерним нападима, који могу изазвати произвољно (византијско) понашање чвора – чвор може слати нетачне, контрадикторне или злонамерне поруке, а да притом изгледа као да нормално функционише.

Practical Byzantine Fault Tolerance (PBFT) [1], који су представили Miguel Castro и Barbara Liskov (2002), први је алгоритам који решава проблем византијског консензуса на начин који је практично применљив – ефикасан је (користи симетричну криптографију уместо скувих операција јавног кључа), безбедан је у асинхроним мрежама попут Интернета (не ослања се на претпоставке о временским границама за безбедност) и толерантан је на до $\lfloor (n-1)/3 \rfloor$ неисправних реплика од укупно n реплика у систему.

Циљ овог пројекта је имплементација симулатора PBFT алгоритма који верно прати нормалан ток протокола (PRE-PREPARE, PREPARE, COMMIT фазе) и механизам промене лидера (VIEW-CHANGE), уз демонстрацију понашања система у присуству различитих типова грешака.

2. Опис PBFT алгоритма

PBFT имплементира state machine replication – сервис се моделује као детерминистичка state машина реплицирана преко n чворова (реплика), где је $n = 3f + 1$, а f је максималан број реплика које могу истовремено бити неисправне. Овај минимални однос $n > 3f$ произлази из захтева да свака два кворума коришћена у протоколу морају имати бар једну исправну реплику у пресеку (пресек кворума + доступност кворума без неисправних реплика).

Реплике пролазе кроз низ конфигурација названих view-ови. У сваком view-у једна реплика је примарна (лидер) и одређује редослед извршавања операција

које захтевају клијенти, док су остале backup реплике. Примарна реплика за view v одређена је формулом $p = v \bmod n$.

Клијент захтева извршење операције multicast поруком REQUEST свим репликама. Свака реплика која изврши операцију шаље одговор директно клијенту; клијент прихвата резултат тек када прими $f + 1$ поклапајућих одговора од различитих реплика (јер је бар једна од њих сигурно исправна).

2.1 Фазе алгоритма

Нормално извршавање протокола одвија се кроз три фазе.

PRE-PREPARE фаза

Примарна реплика, по пријему клијентског захтева, додељује му редни број (sequence number) унутар тренутног view-а и broadcast-ује PRE-PREPARE поруку свим backup репликама. Backup реплика прихвата ову поруку само ако је у одговарајућем view-у, није већ прихватила други PRE-PREPARE за исти редни број са другачијим садржајем (заштита од двоструког предлагања од стране неисправног примарног) и поседује одговарајући REQUEST у свом логу.

PREPARE фаза

Након прихватања PRE-PREPARE-а, реплика broadcast-ује PREPARE поруку осталима. Реплика сматра захтев prepared када поседује prepared сертификат: један PRE-PREPARE и $2f$ поклапајућих PREPARE порука (укључујући сопствену) за исти редни број, view и digest садржаја. Овај сертификат доказује да се кворум реплика сагласио око редоследа додељеног датом захтеву унутар посматраног view-а и гарантује да две различите реплике не могу постати prepared за два различита захтева са истим редним бројем у истом view-у.

COMMIT фаза

По постизању prepared статуса, реплика broadcast-ује COMMIT поруку. Реплика извршава операцију тек када поседује и prepared сертификат и committed сертификат ($2f + 1$ поклапајућих COMMIT порука, укључујући сопствену). Овај двостепени услов (prepared и committed) обезбеђује да реплика никад не изврши

операцију само на основу туђих тврдњи, без сопствене, независне потврде да је захтев исправно припремљен.

VIEW-CHANGE

Уколико бацир реплика посумња да примарна реплика не функционише исправно (не шаље PRE-PREPARE у разумном року након пријема захтева, мерено тајмером), покреће промену view-а слањем VIEW-CHANGE поруке која садржи скупове P (захтеви за које је реплика била prepared у претходним view-овима) и Q (захтеви за које је реплика била pre-prepared).

Нови примарни, након што сакупи кворум VIEW-CHANGE порука потврђених одговарајућим VIEW-CHANGE-ACK порукама, извршава формалну процедуру одлучивања која за сваки редни број бира или захтев који је био на путу ка прихватању у претходном view-у (на основу P/Q доказа) или специјалан NULL захтев уколико такав доказ не постоји. На тај начин гарантује се да се ниједна информација о већ скоро прихваћеним захтевима не изгуби приликом промене лидера, а да се истовремено не „измишља“ садржај који никада није био стварно припремљен код довољно реплика.

3. Архитектура имплементираног симулатора

Симулатор је имплементиран у програмском језику Go, чији су конкурентни примитиви (goroutine и канали) природно погодни за моделовање независних, комуницирајућих процеса.

Структура пројекта:

- `message/` – дефиниције типова порука (REQUEST, PRE-PREPARE, PREPARE, COMMIT, VIEW-CHANGE, VIEW-CHANGE-ACK, NEW-VIEW, REPLY, FETCH-VIEW-CHANGE) и њихових структура (укључујући P/Q скупове за view change)
- `node/` – имплементација реплике (Node), са комплетном PBFT логиком
- `client/` – имплементација клијента који multicast-ује захтеве и чека кворум одговора, са механизмом ретрансмисије
- `network/` – повезује чворове кроз Go канале, управља покретањем и гашењем

Конкурентност

Свака реплика се извршава као независна goroutine (`go node.Run()`), са сопственим стањем (заштићеним mutex-ом за приступ из више goroutine-a) и сопственим улазним каналом порука (Inbox). Свака реплика додатно покреће посебне goroutine-е за тајмере (детекција неисправног примарног) које раде независно од главне петље обраде порука. Ово је неопходно јер реплика мора истовремено чекати на долазне поруке и мерити протекло време, што се не би могло постићи у једној секвенцијалној петљи.

Комуникација

Сва комуникација између чворова реализована је искључиво кроз типизирани поруке које путују Go каналима. Метод Broadcast шаље поруку свим осталим репликама (користи се за PRE-PREPARE, PREPARE, COMMIT, VIEW-CHANGE и NEW-VIEW), док се SendTo користи за комуникацију тачка–до–тачке (VIEW-CHANGE-ACK се шаље искључиво новом примарном, тачно према спецификацији алгоритма).

Симулација грешака

Имплементирана су четири типа грешака: пад чвора (CrashFault – гашење goroutine-е, поруке које му стижу губе се у пуном баферу без блокирања пошљаоца), византијско понашање (ByzantineFault – насумично слање искривљеног садржаја различитим примаоцима, симулирајући equivocation напад), кашњење порука (DelayFault – асинхроно одложено слање, без блокирања обраде долазних порука код пошљаоца) и спор примарни (SlowLeaderFault).

Додатно, механизам SkipPeers омогућава детерминистички (поновљив) сценарио делимичног изостанка порука ка одабраним репликама, коришћен за прецизно демонстрирање преноса P/Q скупова кроз view change.

4. Верификована безбедносна својства

Имплементација је тестирана кроз неколико карактеристичних сценарија који директно илуструју безбедносне гаранције описане у раду:

- Нормалан рад – сви чворови успешно постижу консензус и клијент добија $f + 1$ поклапајућих одговора.

- Equivocation напад примарног – примарна реплика шаље различите садржаје различитим репликама; ниједна реплика не успева да достигне кворум за сопствену (можда искривљену) верзију, тајмери истичу и покреће се view change, све док поштен примарни не преузме и успешно заврши рунду.
- Пренос prepared захтева кроз view change – када примарни падне након што је део реплика већ постао prepared, нови примарни на основу P/Q доказа поново предлаже исти захтев (не губи напредак), тачно према процедури одлучивања.
- NULL fallback – када ниједна реплика није стигла да постане prepared пре промене view-а, процедура одлучивања исправно бира NULL (no-op) за тај редни број, уместо да измишља садржај који никада није био стварно потврђен код довољно реплика.
- Прекорачен број толерисаних грешака – када број неисправних реплика премаше теоријску границу f (нпр. угасе се 2 од 4 реплике при $f = 1$), систем исправно не успева да постигне консензус; клијент не добија одговор ни након поновљеног слања захтева, чиме се демонстрира зашто је услов $n \geq 3f + 1$ неопходан, а не само довољан, за очување безбедности и живости протокола.

5. Позната ограничења и свесна поједностављења

У складу са дозвољеним поједностављењима из спецификације пројекта, имплементација намерно изоставља:

- Криптографску аутентификацију порука (MAC-ови по пару реплика) – поруке се преносе без икаквог потписа или кода за аутентификацију. Ово поједностављење елиминише потребу за улогом VIEW-CHANGE-ACK порука из рада (доказ аутентичности VIEW-CHANGE поруке код баскуп реплике која не може сама да провери њен MAC) – таква не постоји у систему без аутентификације порука.
- Checkpoint/garbage collection механизам – low water mark је фиксиран на 0; ово је оптимизација за дуготрајне системе, а не суштински део безбедности view change протокола.
- Проактивни опоравак (BFT-PR), описан у другом делу рада, није имплементиран јер превазилази обим основног симулатора.
- Ретрансмисија VIEW-CHANGE порука имплементирана је кроз експлицитну FETCH-VIEW-CHANGE поруку.

6. Закључак

Имплементирани симулатор успешно демонстрира суштинске механизме PBFT алгоритма – трофазну размену порука за постизање консензуса у нормалном раду и view change протокол са формалном процедуром одлучивања за опоравак од неисправног лидера.

Кроз серију контролираних сценарија показано је да имплементација исправно прати формалне дефиниције prepared и committed сертификата из оригиналног рада, као и безбедносну гаранцију да се view change протоколом никад не губи већ постигнут напредак, нити се измишља садржај који није био стварно потврђен.

Рад на пројекту је додатно потврдио кључну улогу прецизног дефинисања кворума ($2f$ наспрам $2f + 1$, услови A1/A2/B из процедуре одлучивања) за коректност и толеранцију на грешке система минималне величине ($n = 3f + 1$).

7. Литература

- [1] M. Castro, B. Liskov, "Practical Byzantine Fault Tolerance and Proactive Recovery", ACM Transactions on Computer Systems, vol. 20, no. 4, pp. 398–461, Nov. 2002. DOI: 10.1145/571637.571640.